Understanding Codebase

Thursday, February 5, 2026 2:23 PM

Backend FASTAPI:

- Uvicorn server running on http://0.0.0.0.8000
- FASTAPI run its startup events (connect to db, initialize caches)
- Langwatch.client : instrumentation/tracing registerd to flush traces when the process exits
- Front use http://localhost:8000 when making API requests

Note: Test reachability quickly with curl http://192.168.0.152:8000/docs

192.168.0.152:62263 - "GET /docs HTTP/1.1" 200 OK

First string of numbers is client that made the request
Next we have a GET for the docs path
HTTP protocol verison used
200 ok server successfully returned the page (no erro)

Retrieval System:

- Uses multi-layered retrieval architecuture centered around vector similarity search
  - Using Facebook AI similarity search (FAISS) db
    - Converts legal documents into dense vector embeddings using Azure Open AI embedding model
- Basic Similarity Search: nearest neighbor approach
- Parent Document Retrieval: to prevent loss of critical legal context. This method retrieves the entrie parent document
- Multi-query/query Rewriting: rephrase user query to deal with different or ambigous phrasing
  - Reformulates user query into multiple variations and searches across them all to ensure a more comprehensive recall of relevant information
- Contextual compression: method filters and compresses retrieved documents to extract only the most pertinent paragraphs

- Retrival logic is trailored to source of the data
  - Rets (main laws) , ast (administrative decisions) and db_doms (court decisions)

- All methods are coordinated by the langchain framework

PM2:

- Is a Node.js process manager used to keep services (like RAG API, indexer, or UI) running in production,handle crash restarts and do simple load balancing

Selenium:

- Open source browser automation framework, often used to test RAG frontend admin tools end to end by driving a real browser and simulating user interaction

HTTPS:

- Over TLS/SSL encryption, in a RAG system it's the secure protocol you use for API Calls (chat endpoint, document upload endpoint) to protect prompts retrieved content, and user data in transit

Endpoints:

- **Concrete HTTP paths and methods** you expose from your services, such as POST/chat, POST/ingest, or GET/health, that client call to interact with the RAG systems capabilities

Normalization:

- Preprocessing step where you rescale feature values (token counts, scores, or metadata features) into a consistent range (0-1) so that no single feature dominates similarity calcualtions or downstream models

Caches:

- Means data or computation results (retrieved passages, embeddings for a document, or model's answer to a common user query) that is calcualed once and in faster temp storage so it can reused quickly

Scoped:

- Dependency injection a scoped service is created once per scope and resused within that request; in a RAG API this might be per-request context object that tracks user, trace IDs, and request local state across components handling that request

Transient:

- A transient service is created every time its requested; in a RAG system this might be a light weight helper like a one-off query builder or DTO mapper that does not need to share state

VMs:

- Virtual machines are virtualized servers you run you RAG components on (vector db, embedding workers, API servers, orchestrators)
- ***Each VM provides isolated compute, memory, storgage, managed by hypervisor in your cloud***

Testing Overview:

Unit & integration testing:

- Test core functionality in isolation
  - Uploading, searching, indexing
- Use pytest
- Whats tested:
  - Document chunking -> splitting PDFs into searchable chuncks
  - FAISS indexing (vector db creation)
  - User isolation no data leakage
  - Upload limits
  - Metadate preservarvation

CI/CD auto tests:

- Automated qualitiy gates before deployment
- Blocks deployment if test fail
- Runs on every merge request

End-to-End tests:

- Complete user journey (upload->search-> output)
- Uses Playwright brower automation
  - Tests communication with FastAPI server HTTP + websocket

- Written in typescript
- Tests user can log in
- File upload works
- Results are returned in real-time (websocket)
- Upload limits are enforced

Quality Metrics:

- Retrieval quality:
  - Tests verify documents are found correctly
- Generation Quality:
  - Langwatch
    - Response accuracy
    - Latency response fast?
    - Token usage, over or under budget
    - User feedback